Coding Assignments #7 (Part Two)

August 4, 2025

Directions:

1. You will be penalized for not following each of the directions exactly as written below. If you aren't sure about what something means, ask the professor!
2. Put all your work in a single Haskell file.
3. Submit your work (before the deadline) to Canvas.
4. **Important: You must avoid global helper functions**. Keep the scope of your helper functions limited by using `let`s and `where`s.
5. **Important:** You will be graded on excessive use of helper functions when patterns or guards would make more sense.
6. **Important:** You will lose points **each time** you use the `if` keyword. Use patterns and guards instead!

1 Background

A trie is a data structure for storing lists of items. In tries, lists are stored so that common prefixes among lists are only stored once. Figure 1 gives an example. At first (1), the trie is empty. Then (2) "a" is inserted. A true in a node indicates that a value is stored in the trie. **Important:** notice that the node containing the 'a' does not have it's boolean value set to `True`, but its *child* (the one that corresponds to 'a') does. After the insertion of "ask" (3) the trie has several nodes. **Important:** The values stored in the trie can be reconstructed by following the path from the root to a node storing `True`. For example, at this stage, "ask" can be reconstructed by following the nodes containing 'a', 's', 'k', and its child. The next insertion is of the empty string, "" (4). Since the empty string does not contain any letters at all, the root of the trie is simply set to `True`. **Important:** When reconstructing/storing values in a trie, always consider that a `True` stored in a node really means the empty string *prefixed by the path to that node* is the value being stored/reconstructed. For example, the empty string can be constructed by just looking at the root and

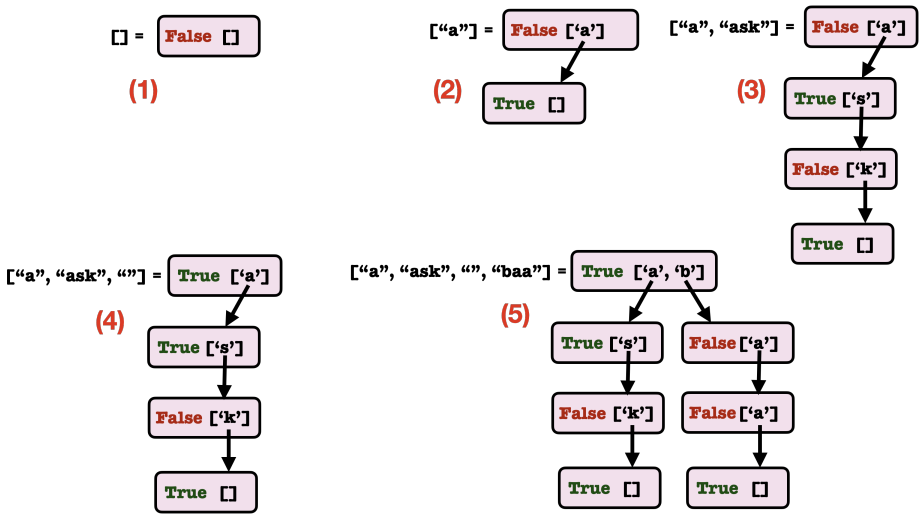

Figure 1: Showing the steps in insert "a", "ask", "", and "baa" into a Trie.

noticing that its boolean value is set to `True`: the prefix in this case is nothing (there are no nodes above the root), so the `True` in the root node indicates that the empty string is being stored. Similarly, the child of the root which also has a `True` boolean value indicates that it is storing the empty string prefixed by the path to it (just "a"), so the child of the root stores "a". Finally, (5) "baa" is inserted. The node that stores "baa" will store the empty string prefixed by "baa". For this to happen, that node can't be in the same path as "a" and "ask" because they both start with "a". So a new child of the root is created which is associated with "b".

In Haskell, these five tries are represented as follows (some lines needed to be broken because of length):

```
T False []                                              (1)
T False [('a', T True [])]                              (2)
T False [('a', T True
          [('s', T False [('k', T True [])])])]         (3)
T True [('a', T True
          [('s', T False [('k', T True [])])])]         (4)
T True [('a', T True
          [('s', T False [('k', T True [])])]),
          'b', T True
          [('a', T false [('a', T True [])])])]         (5)
```

2 Requirements

1. Call your data type **Trie**). It will have a single constructor that has two components: (1) a boolean value; (2) a list of (key, value) associates where the key is an element in the lists the trie stores and the value is a child Trie.

2. (Call your function **trieInsert**). Using only techniques from chapters 1-7b, write a function to insert values into a trie.

3. (Call your function **elof)**. Using only techniques from chapters 1-7b, write a function to test whether a value is in a trie.

4. (Call your function **makeTrie**). Using only techniques from chapters 1-7b, write a function that creates a trie from a list of values.

5. (Call your function **makeTrie**). Using only techniques from chapters 1-7b, write a function that creates a list of values from a trie.

6. Export your module and these functions. You may use helper functions and constructors, but do not export them.